

Network Security

Network Security Report

GitHub Link : <https://github.com/dev-69/secure-chat-laughing-engine>

Phase 1 – Baseline Chat Application

1. System Overview

Phase 1 implements a basic one-to-one chat application using TCP sockets. The system consists of two programs:

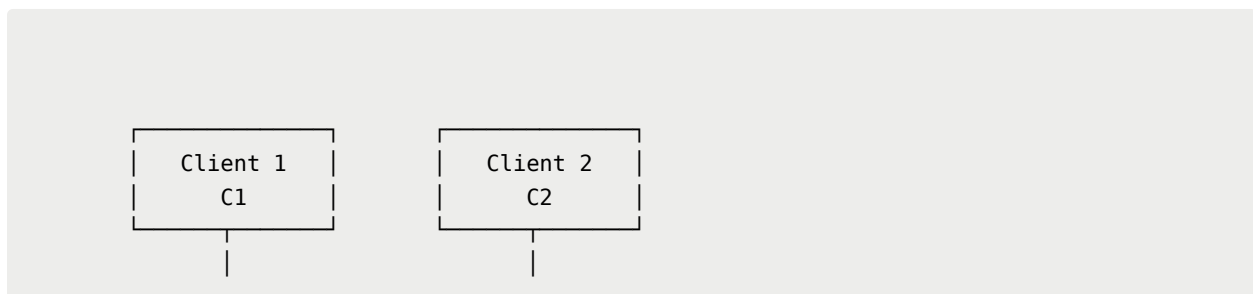
- `server.cpp` – handles client connections and forwards messages.
- `client.cpp` – provides the interface used by each client.

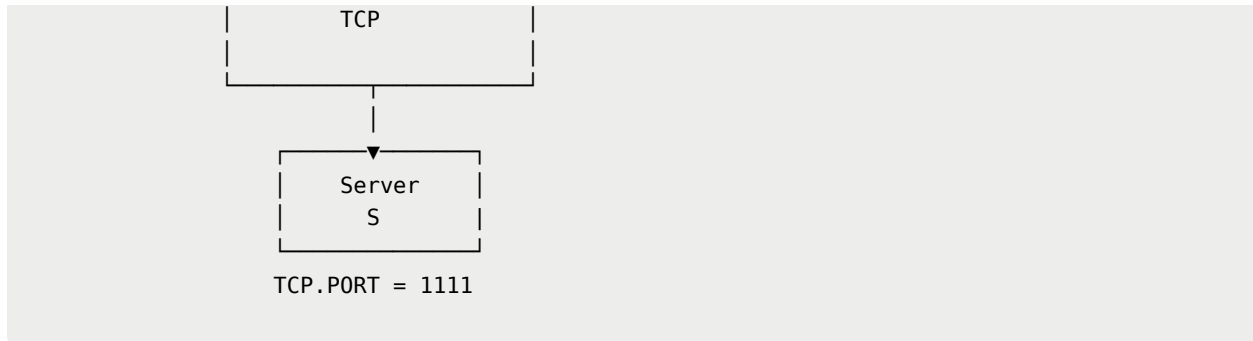
The server listens for TCP connections on **port 1111**. Two clients can connect to the server at the same time and communicate through it.

The application was tested using **three separate physical laptops connected to the same network**. One laptop was used as the server and the other two were used as clients.

2. System Architecture

The communication follows the architecture below:





The server keeps track of connected clients and forwards each message to the intended recipient.

3. Server Implementation

The server creates a TCP socket, binds it to port `1111`, and starts listening for incoming connections.

Each new client connection is handled using a separate thread:

```
std::thread clientThread(handleClient, clientSocket);
clientThread.detach();
```

This allows multiple clients to stay connected and communicate concurrently.

The server maintains a mapping between usernames and socket descriptors:

```
std::unordered_map<std::string, int> clients;
```

For example:

```
client1 → socket descriptor
client2 → socket descriptor
```

Since multiple threads can access this mapping, a mutex is used to protect it from concurrent access.

4. Message Routing

Messages can be sent using the following format:

```
@username message
```

For example:

```
@client2 Hello from client1
```

The client extracts the destination username and sends the message to the server. The server looks up that username in its client map and forwards the message to the corresponding socket. The server also prints successfully relayed messages. For example:

```
[CHAT] client1 -> client2 : Hello from client1
```

This is also useful for demonstrating the security problem in Phase 1: the server has direct access to the complete message because the message is still plaintext.

5. Client Commands

The client supports the required commands:

```
@username message
```

```
/chat username
```

```
/who
```

```
/quit
```

```
/chat
```

Sets the current chat partner so that the username does not need to be specified for every message.

/who

Requests the list of currently connected users from the server.

/quit

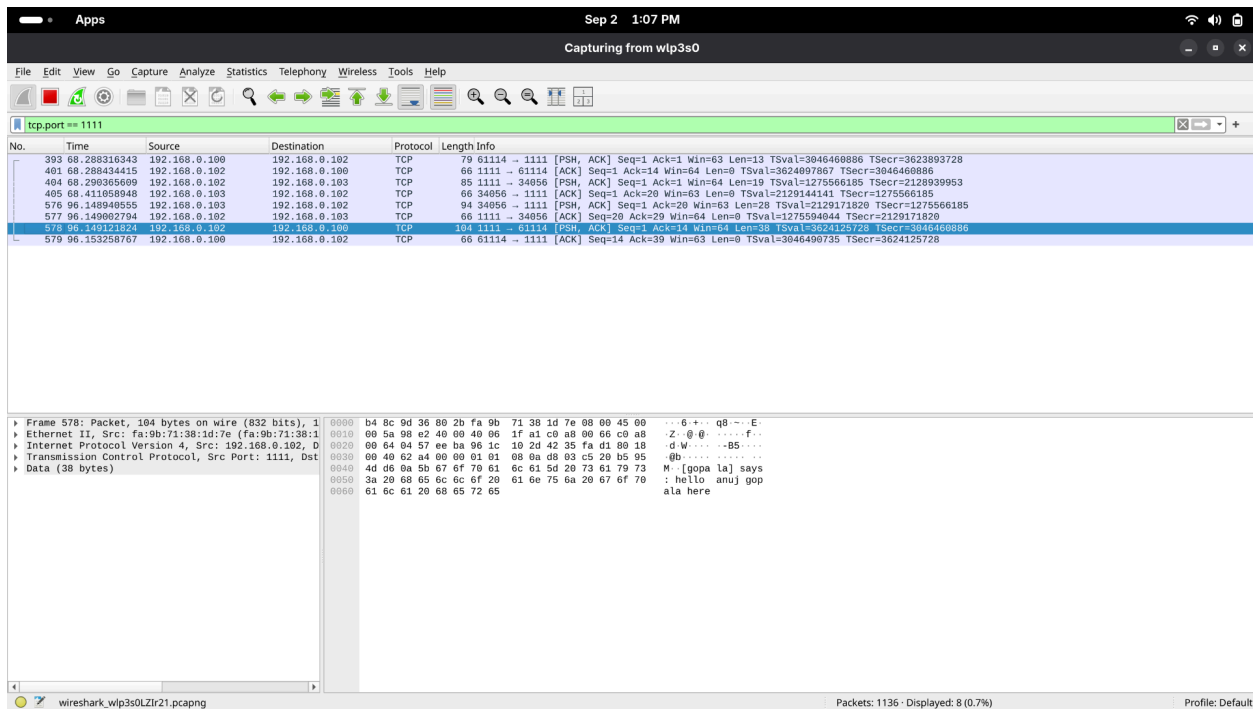
Terminates the client connection cleanly.

6. Plaintext Verification

No encryption is used in Phase 1. Messages are therefore transmitted directly over the TCP connection.

The server logs already show the message contents in plaintext. To verify that the same information is visible on the network, traffic was captured using **Wireshark**.

Figure 1 – Wireshark TCP Stream Showing Plaintext Communication



The capture confirms that someone monitoring the network traffic can read the exchanged messages without needing any decryption.

Phase 2 – Client-Server Confidentiality Using Diffie-Hellman

1. Overview

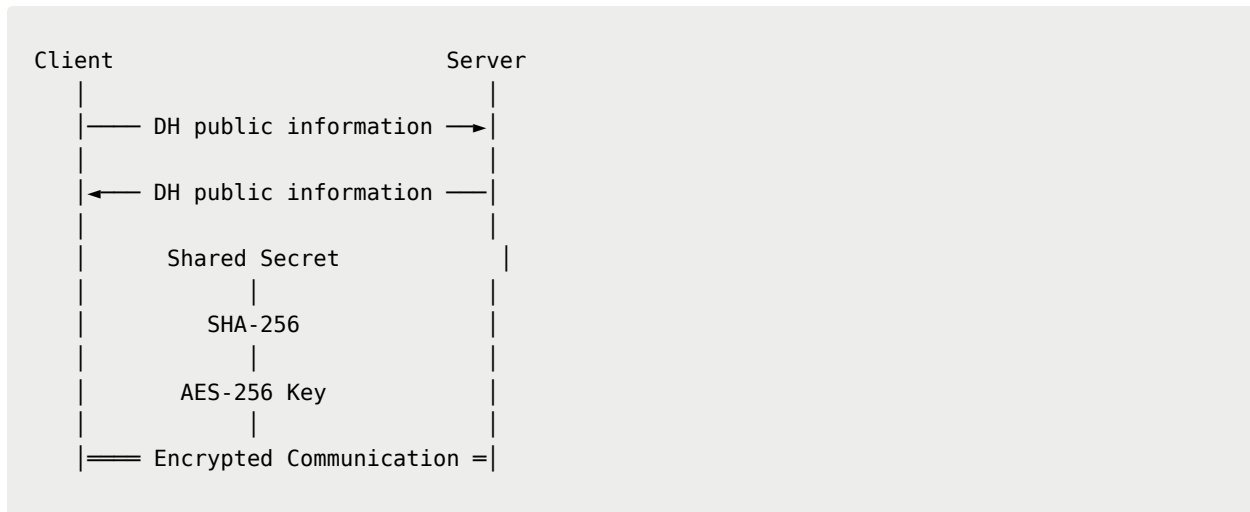
Phase 2 adds confidentiality to the Phase 1 chat application.

A manually implemented **Diffie-Hellman (DH)** key exchange is used to establish a shared secret between each client and the server. The shared secret is then processed using **SHA-256** to derive a 256-bit AES key.

Messages are subsequently encrypted using **AES-256-GCM**.

The DH implementation uses OpenSSL's low-level `BIGNUM` operations and modular exponentiation rather than a built-in DH/ECDH implementation.

The basic flow is:



The same process is performed independently for every client.

2. Diffie-Hellman Implementation

The DH functionality is implemented in `dh.cpp`.

The implementation uses the RFC 3526 2048-bit MODP Group (Group 14).

The generator is set to:

```
BN_set_word(g, 2);
```

A private value is generated for each `DHE` instance:

```
BN_rand_range(priv_key, p);
```

The corresponding public value is calculated using modular exponentiation:

```
BN_mod_exp(pub_key, g, priv_key, p, ctx);
```

For a client and server, the exchange can be represented as:

Client:

```
private = a  
public  =  $g^a \bmod p$ 
```

Server:

```
private = b  
public  =  $g^b \bmod p$ 
```

The client computes:

```
 $(g^b)^a \bmod p$ 
```

while the server computes:

```
 $(g^a)^b \bmod p$ 
```

Both calculations produce the same shared secret.

The secret itself is never transmitted over the network.

3. Independent Key for Each Client

The server creates a separate `DHE` object for every client connection:

```
void handleClient(int clientSocket) {  
  
    DHE dh;  
  
    dh.generateKeys();  
  
}
```

```
...  
  
}
```

Since `handleClient()` is executed separately for each client, every client performs its own DH exchange with the server.

The server also stores the corresponding `DHE` object along with the client's socket:

```
struct ClientData {  
  
    int socket;  
  
    DHE* dh_ptr;  
  
};
```

This allows the server to use the correct encryption key for each connection.

Conceptually:

```
Client 1 ——— Key K1 ——— Server  
Client 2 ——— Key K2 ——— Server  
  
K1 ≠ K2
```

Thus, the two client connections do not share the same symmetric encryption key.

4. Deriving the AES Key

The raw DH shared secret is not directly used as the AES key.

After calculating the shared secret, it is converted into bytes and hashed using SHA-256:


```

EVP_DigestInit_ex(mdctx, EVP_sha256(), nullptr);

EVP_DigestUpdate(
    mdctx,
    secret_bytes.data(),
    secret_bytes.size()
);

EVP_DigestFinal_ex(
    mdctx,
    aes_key.data(),
    &hash_len
);

```

SHA-256 produces 32 bytes:

$32 \text{ bytes} \times 8 = 256 \text{ bits}$

The resulting value is therefore used as the AES-256 key.

The overall process is:



Using SHA-256 also gives a fixed-length representation of the DH shared secret that can be directly used as a symmetric key.

5. AES-GCM Encryption

Once the DH exchange is complete, communication is encrypted using **AES-256-GCM**.

A new 12-byte random nonce is generated for every message:

```
unsigned char nonce[12];

RAND_bytes(nonce, sizeof(nonce));
```

AES-256-GCM is then initialized using the derived key and nonce:

```
EVP_EncryptInit_ex(
    ctx,
    EVP_aes_256_gcm(),
    nullptr,
    aes_key.data(),
    nonce
);
```

The encrypted message contains the nonce, authentication tag, and ciphertext:

Nonce 12 bytes	Authentication Tag 16 B	Ciphertext
-------------------	----------------------------	------------

The GCM authentication tag allows the receiver to detect if the encrypted data has been modified.

Therefore, AES-GCM provides both:

- Confidentiality
- Integrity/authentication of the ciphertext

6. Encrypted Login and Chat Communication

The username exchange is also protected after the DH handshake.

For example, the server encrypts its username prompt:

```
std::vector<unsigned char> enc_prompt = dh.encrypt(prompt);
```

The client decrypts the prompt using the derived AES key.

The username entered by the client is also encrypted before being sent:

```
std::vector<unsigned char> enc_username = dh.encrypt(username);
```

Normal application messages and commands are also encrypted, including:

```
@client2 Hello  
  
/who  
  
/quit
```

Therefore, after the initial DH exchange, the actual application data is not transmitted as plaintext.

7. DH Fingerprint Verification

A fingerprint is generated from the derived AES key to make it easier to verify that both sides obtained the same key.

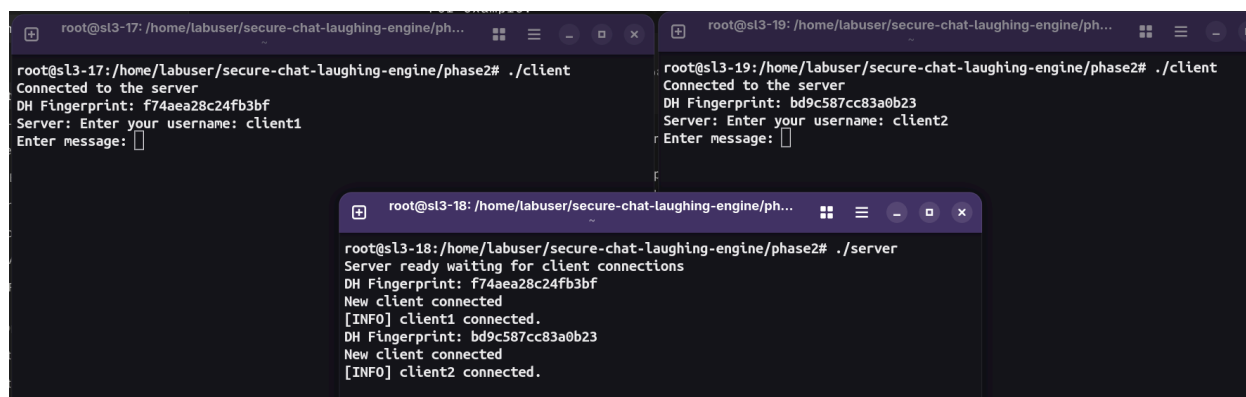
The fingerprint is produced by hashing the AES key and displaying the first eight bytes of the resulting hash.

The actual shared secret and AES key are never printed.

During testing, the fingerprint displayed by the client matched the fingerprint displayed by the server for the corresponding connection.

This provides evidence that both endpoints independently derived the same symmetric key.

Figure 2 – Matching DH Fingerprints



```
root@sl3-17: /home/labuser/secure-chat-laughing-engine/phase2# ./client
Connected to the server
DH Fingerprint: f74aea28c24fb3bf
Server: Enter your username: client1
Enter message:

root@sl3-19: /home/labuser/secure-chat-laughing-engine/phase2# ./client
Connected to the server
DH Fingerprint: bd9c587cc83a0b23
Server: Enter your username: client2
Enter message:

root@sl3-18: /home/labuser/secure-chat-laughing-engine/phase2# ./server
Server ready waiting for client connections
DH Fingerprint: f74aea28c24fb3bf
New client connected
[INFO] client1 connected.
DH Fingerprint: bd9c587cc83a0b23
New client connected
[INFO] client2 connected.
```

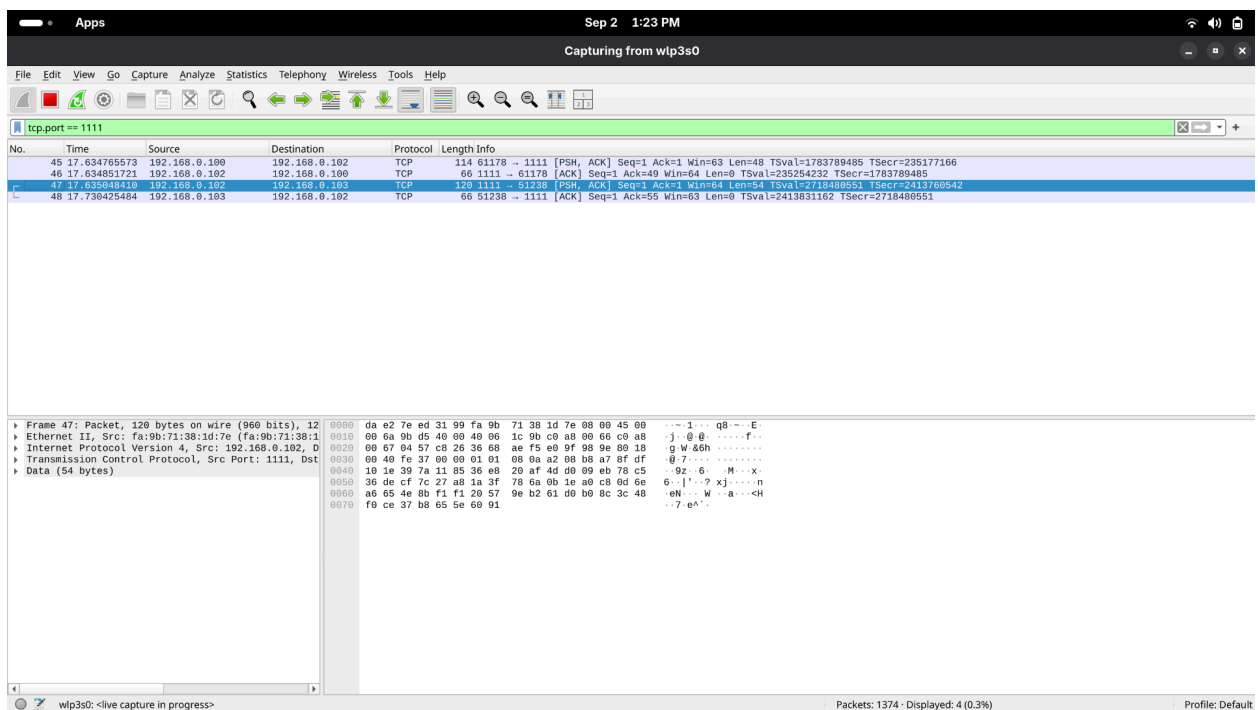
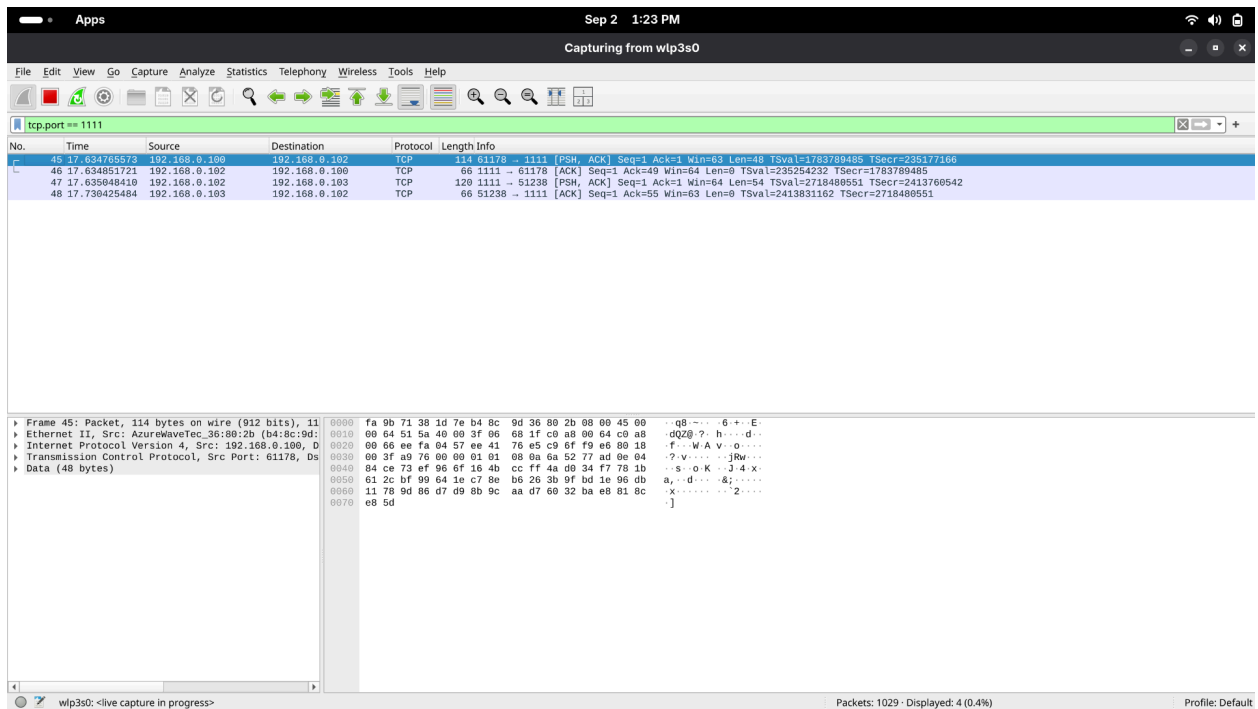
8. Wireshark Verification

The Phase 1 traffic capture was repeated after adding encryption.

The traffic was captured using Wireshark on TCP port 1111 and inspected using:

In Phase 2, the application data appears as binary-looking encrypted data rather than readable chat messages.

Figure 3 – Wireshark Showing Encrypted Phase 2 Traffic



This shows that a passive observer monitoring the network can no longer directly read the chat messages from the captured TCP traffic.

9. AES-GCM Tamper Detection

AES-GCM also protects the integrity of the encrypted message.

To test this, one byte of the ciphertext was modified before attempting to decrypt it.

The decryption process verifies the GCM authentication tag:

```
EVP_CIPHER_CTX_ctrl(  
    ctx,  
    EVP_CTRL_GCM_SET_TAG,  
    16,  
    tag  
);
```

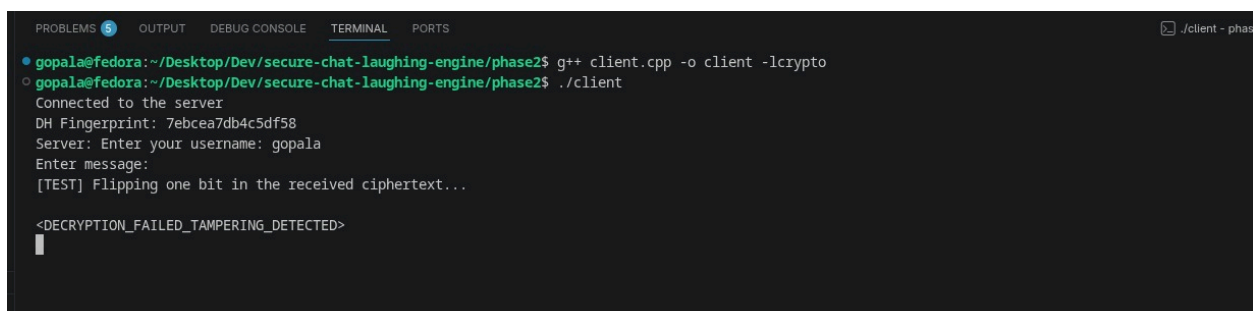
If the ciphertext has been modified, authentication fails and the message is rejected:

```
if (ret <= 0) {  
    return "<DECRYPTION_FAILED_TAMPERING_DETECTED>";  
}
```

The observed output was:

```
<DECRYPTION_FAILED_TAMPERING_DETECTED>
```

Figure 4 – AES-GCM Tampering Test



```
PROBLEMS 5 OUTPUT DEBUG CONSOLE TERMINAL PORTS .client - phas  
gopala@fedora:~/Desktop/Dev/secure-chat-laughing-engine/phase2$ g++ client.cpp -o client -lcrypto  
gopala@fedora:~/Desktop/Dev/secure-chat-laughing-engine/phase2$ ./client  
Connected to the server  
DH Fingerprint: 7ebcea7db4c5df58  
Server: Enter your username: gopala  
Enter message:  
[TEST] Flipping one bit in the received ciphertext...  
  
<DECRYPTION_FAILED_TAMPERING_DETECTED>  
█
```

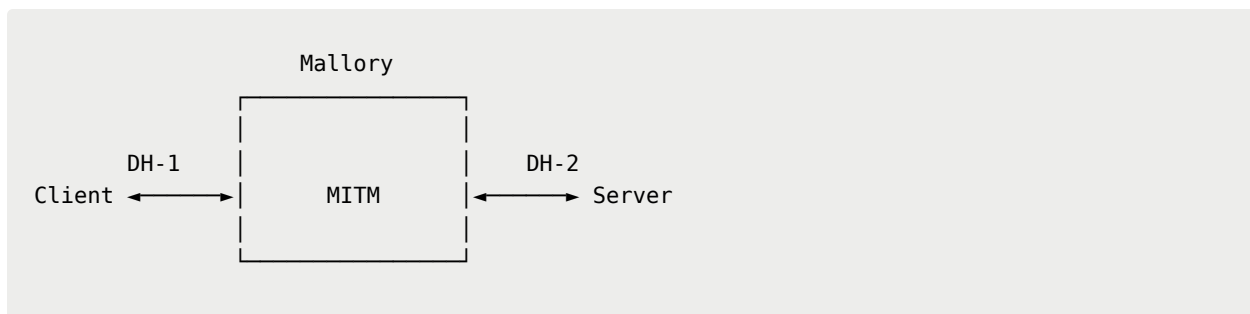
The test confirms that modification of the encrypted data is detected rather than being silently accepted.

10. Man-in-the-Middle Attack

Although DH protects against passive eavesdropping, the basic DH exchange does not authenticate who is participating in the exchange.

To demonstrate this limitation, a separate `mitm.cpp` proxy was implemented.

The proxy performs two independent DH exchanges:



This results in two different keys:

Client ↔ Mallory : Key K1

Mallory ↔ Server : Key K2

The client believes it has established a secure connection with the server, while the server believes it has established a secure connection with the client.

Mallory, however, has access to both keys.

11. MITM Message Interception

The proxy decrypts messages received from the client using the client-facing key:

```
std::string plaintext =  
    dh_client->decrypt(payload);
```

The decrypted message is then logged:

```
std::cout << "\n[INTERCEPT C->S]: "  
    << plaintext << std::endl;
```

Mallory then encrypts the plaintext using the server-facing key and forwards it to the real server.

The same process happens for messages travelling from the server back to the client.

The attack can therefore be represented as:

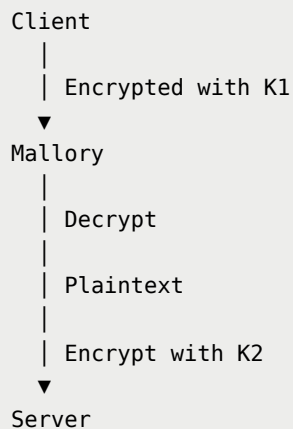


Figure 5 – MITM Proxy Intercepting Plaintext


```

dibyansh@LAPTOP-3AF6P4N9:/mnt/c/Users/HP/Desktop/Assignments/dev - network/secure-chat-laughing-engine/phase2$ ./client
Connected to the server
DH Fingerprint: 436dca84a9403d70
Server: Enter your username: dibyansh_gupta
Enter message: /who
Enter message:

--- Online Users ---
- dibyansh_gupta
- anuj_ramane
-----

/chat anuj_ramane
Chat partner set to anuj_ramane
Enter message: hello anuj how are you
Enter message:

```

```

75     ... DHE dh_server_facing;
76     ... DHE dh_client_facing;

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
hedwig69@fedora:~/Desktop/IIT-Bombae/Semester3/CS6008 - Network Security/Secure_Chat_Assignment/phase2$ ./server
Server ready waiting for client connections
DH Fingerprint: 436dca84a9403d70
New client connected
DH Fingerprint: 77ad40861c2dcfee
New client connected
[INFO] anuj_ramane connected.
[INFO] dibyansh_gupta connected.
[CHAT] dibyansh_gupta -> anuj_ramane : hello anuj how are you

```

```

105     std::thread t1(relay_client_to_server, client_fd, server_fd, &dh_client_facing, &dh_server_facing);
106

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
gopala@fedora:~/Desktop/Dev/secure-chat-laughing-engine/phase2$ ./
client      client_normal client_victim mitm      server
gopala@fedora:~/Desktop/Dev/secure-chat-laughing-engine/phase2$ ./mitm
[MITM] Connected to real Server.
[MITM] Waiting for victim client on port 9999...
[MITM] Victim Client connected!
[MITM] Handshakes complete. Logging traffic...

[INTERCEPT S->C]: Server: Enter your username:

[INTERCEPT C->S]: anuj_ramane

[INTERCEPT S->C]:
[dibyansh_gupta] says: hello anuj how are you

```

```

Problems 6 Output Debug Console Terminal Ports
ws1 - pha

anujr@AnujASUS:/mnt/c/Users/anujr/Downloads/dev/secure-chat-laughing-engine/phase2$ g++ client.cpp -o client -lcrypto
anujr@AnujASUS:/mnt/c/Users/anujr/Downloads/dev/secure-chat-laughing-engine/phase2$ ./client
Connected to the server
DH Fingerprint: 40d26dba87280c38
Server: Enter your username: anuj_ramane
Enter message:

[dibyansh_gupta] says: hello anuj how are you

```

This shows that unauthenticated DH alone is vulnerable to an active MITM attack.

12. Phase 2 Result

Phase 2 adds confidentiality and authenticated encryption to the chat application.

Each client independently performs a DH exchange with the server and derives its own AES-256 key. Messages are then protected using AES-256-GCM.

The following tests were performed:

- Matching DH fingerprints on both endpoints
- Wireshark inspection of encrypted traffic
- AES-GCM ciphertext tampering
- MITM interception using the Phase 2 proxy

The Wireshark capture confirms that the application messages are no longer visible as plaintext, and the tampering test shows that modified ciphertext is rejected.

However, the MITM experiment exposes a limitation of plain DH: **the protocol establishes a shared secret but does not prove the identity of the other party.**

This is the problem addressed in Phase 3.

Phase 3 – Server Authentication Using PKI

Objective

Phase 3 adds server authentication using **Public Key Infrastructure (PKI)**.

The client now verifies that it is communicating with the legitimate server before continuing with the Diffie-Hellman exchange.

The authentication process has two parts:

1. Certificate validation
2. Proof-of-possession of the server's private key

Only after both checks succeed does the client continue with the DH exchange.

1. Certificate Authority and Server Certificate Setup

A private Certificate Authority was created using OpenSSL.

The CA consists of:

- A private CA key
- A self-signed CA certificate

A separate server key pair was then generated. A Certificate Signing Request (CSR) was created from the server public key and signed by the CA.

The resulting files are:

```
certs/
├── ca.key
├── ca.crt
├── server.key
└── server.crt
```

The following commands were used.

Generate the CA private key

```
openssl genrsa -out certs/ca.key 4096
```

Generate the self-signed CA certificate

```
openssl req -x509 -new -nodes \  
    -key certs/ca.key \  
    -sha256 -days 3650 \  
    -out certs/ca.crt \  
    -subj "/C=IN/ST=Maharashtra/L=Mumbai/O=SecureChat/OU=CA/CN=SecureChat Root CA"
```

Generate the server private key

```
openssl genrsa -out certs/server.key 2048
```

Generate the server CSR

```
openssl req -new \  
    -key certs/server.key \  
    -out certs/server.csr \  
    -subj "/C=IN/ST=Maharashtra/L=Mumbai/O=SecureChat/OU=Server/CN=SecureChat Server"
```

Sign the server CSR using the CA

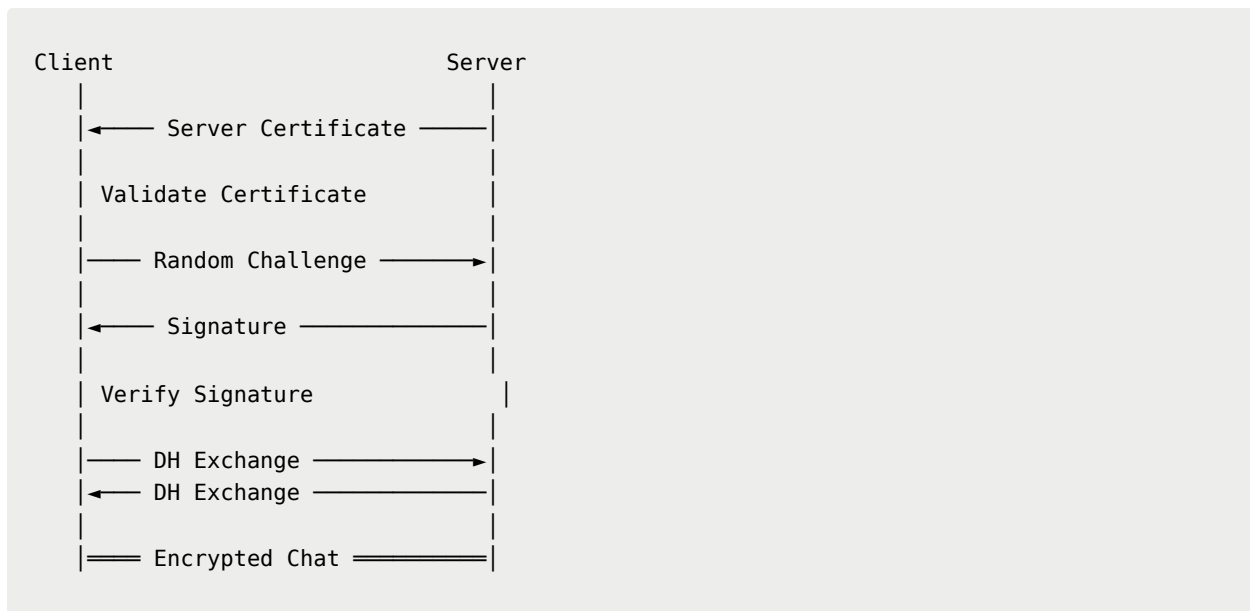
```
openssl x509 -req \  
    -in certs/server.csr \  
    -CA certs/ca.crt \  
    -CAkey certs/ca.key \  
    -CAcreateserial \  
    -out certs/server.crt \
```

```
-days 365 \  
-sha256
```

2. Updated Client and Server

The Phase 2 client and server were modified to add certificate authentication before the DH exchange.

The new handshake is:



2.1 Certificate Validation

Before performing DH, the server sends its certificate to the client.

The client receives the certificate and validates it against the trusted CA certificate:

```
certs/ca.crt
```

The client checks that the certificate is:

- Signed by the trusted CA
- Within its validity period
- Valid according to the configured trust store

If certificate validation fails, the client closes the connection and does not continue with the DH exchange.

This prevents an attacker from simply presenting an arbitrary certificate to the client.

2.2 Proof-of-Possession

Certificate validation alone is not sufficient.

An attacker could potentially obtain a copy of the legitimate server certificate. However, the attacker should not be able to authenticate without the corresponding private key.

To test this, the client generates a random 32-byte challenge:

```
Client → Server : Random Challenge
```

The legitimate server signs the challenge using:

```
certs/server.key
```

The signature is sent back to the client:

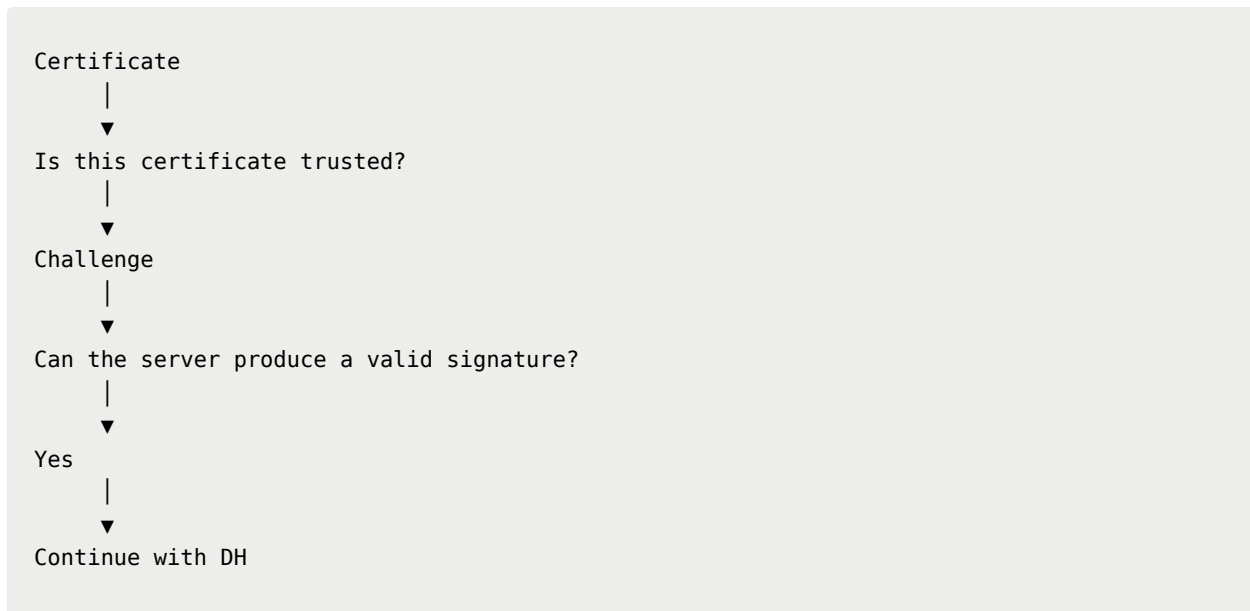
```
Client → Server : Challenge
```

```
Client ← Server : Signature(Challenge)
```

The client extracts the public key from the already-validated server certificate and uses it to verify the signature.

Only if the signature verification succeeds does the client continue with the DH exchange.

In other words:



2.3 Legitimate Phase 3 Connection

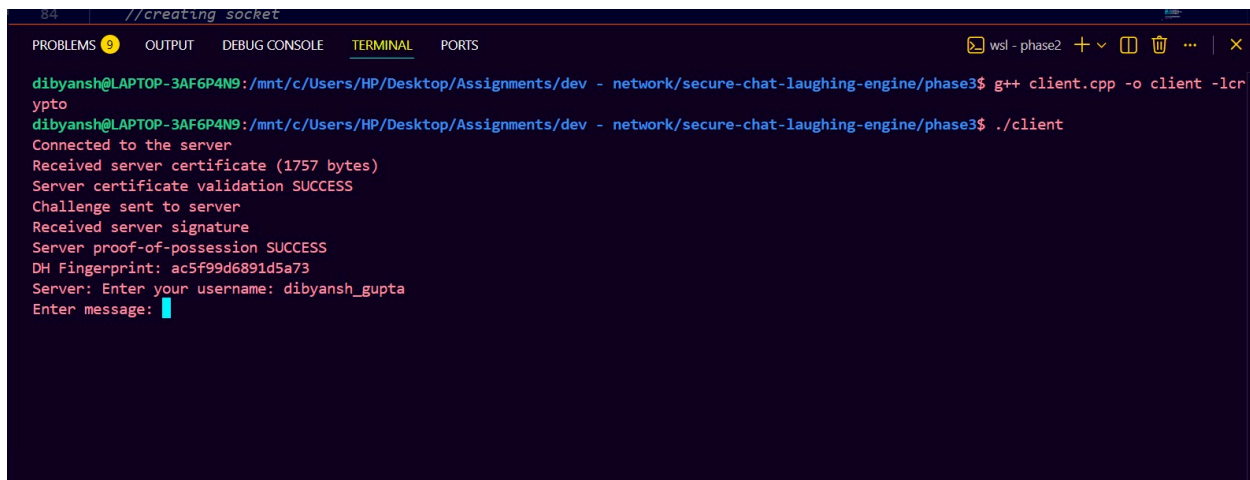
A legitimate connection was tested using the real server certificate and corresponding private key.

The client output showed successful certificate validation and proof-of-possession before the DH exchange continued.

Figure 7 – Legitimate Phase 3 Connection

```
hedwig69@fedora:~/Desktop/IIT-Bombay/Semester3/CS6008 - Network Security/Secure_Chat_Assignment/phase3$ ./server
Server ready waiting for client connections
Failed to receive challenge
Challenge received from client
Challenge signed and signature sent to client
DH Fingerprint: ac5f99d6891d5a73
New client connected
[INFO] dibyansh_gupta connected.
```

The screenshot shows a terminal window with the output of the `./server` command. The output indicates that the server is ready, receives a challenge, signs it, and sends it to the client. It also shows the DH fingerprint and that a new client (dibyansh_gupta) has connected.



```
84 //creating socket
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
dibyansh@LAPTOP-3AF6P4N9:/mnt/c/Users/HP/Desktop/Assignments/dev - network/secure-chat-laughing-engine/phase3$ g++ client.cpp -o client -lcr
ypt
dibyansh@LAPTOP-3AF6P4N9:/mnt/c/Users/HP/Desktop/Assignments/dev - network/secure-chat-laughing-engine/phase3$ ./client
Connected to the server
Received server certificate (1757 bytes)
Server certificate validation SUCCESS
Challenge sent to server
Received server signature
Server proof-of-possession SUCCESS
DH Fingerprint: ac5f99d6891d5a73
Server: Enter your username: dibyansh_gupta
Enter message:
```

The screenshot should show output similar to:

```
Received server certificate
Server certificate validation SUCCESS
Challenge sent to server
Received server signature
Server proof-of-possession SUCCESS
DH Fingerprint: ...
```

3. Updated MITM Attack

The MITM proxy from Phase 2 was modified to attack the Phase 3 protocol.

In the first attack, Mallory generates a different certificate that is **not signed by the trusted CA**.

Mallory sends this certificate to the victim client.

The client attempts to validate the certificate using:


```
certs/ca.crt
```

Since Mallory's certificate was not issued by the trusted CA, certificate validation fails.

The client then closes the connection before the DH exchange can take place.

The expected behaviour is:

Figure 8 – MITM Attack Fails Due to Invalid Certificate

```
anujr@AnujASUS:/mnt/c/Users/anujr/Downloads/dev/secure-chat-laughing-engine/phase3$ ./client
Connected to the server
Received server certificate (1757 bytes)
Server certificate validation SUCCESS
Challenge sent to server
Received server signature
SERVER PROOF-OF-POSSESSION FAILED
anujr@AnujASUS:/mnt/c/Users/anujr/Downloads/dev/secure-chat-laughing-engine/phase3$
```

4. Proof-of-Possession Bypass Attempt

A second attack was performed to test whether an attacker could bypass authentication simply by obtaining a copy of the legitimate server certificate.

In this attack, Mallory possesses:

```
server.crt
```

but does **not** possess:

```
server.key
```

Mallory forwards the legitimate server certificate to the client. Because the certificate is valid and trusted, the certificate validation step succeeds.

The client then generates a random challenge.

Mallory attempts to sign the challenge using a different private key.

The client verifies the received signature using the public key contained in the legitimate server certificate.

Since Mallory's private key does not correspond to the public key in the certificate, verification fails.

The client therefore rejects the connection.

Expected output:

```
Received server signature  
  
SERVER PROOF-OF-POSSESSION FAILED
```

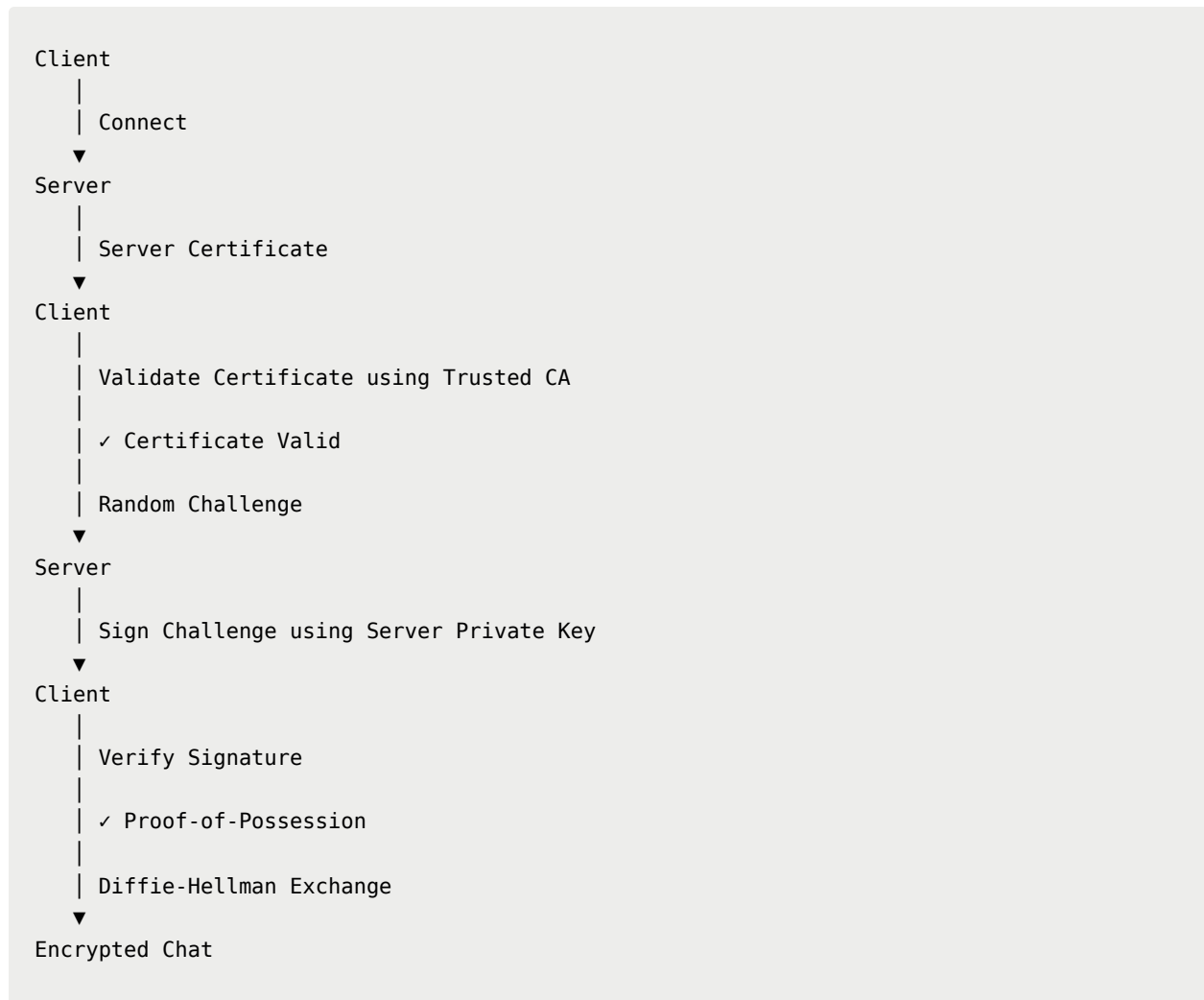
Figure 9 – Proof-of-Possession Bypass Attempt Rejected

```
gopala@fedora:~/Desktop/Dev/secure-chat-laughing-engine/phase3$ ./mitm_adv  
[MITM] Connected to real server.  
[MITM] Waiting for victim client on port 9999...  
[MITM] Victim client connected!  
[MITM] Captured legitimate server certificate (1757 bytes)  
[MITM] Forwarded legitimate certificate to victim.  
[MITM] Loaded Mallory's DIFFERENT private key.  
[MITM] Received client's challenge.  
[MITM] Signed challenge using WRONG private key.  
[MITM] Sent bogus signature to victim.  
[MITM] Victim closed the connection.  
[MITM] Proof-of-possession successfully blocked the attack!  
gopala@fedora:~/Desktop/Dev/secure-chat-laughing-engine/phase3$
```

This test is important because it shows that simply possessing a valid certificate is not enough. The server must also demonstrate possession of the corresponding private key.

5. Phase 3 Security Flow

The final Phase 3 handshake can be summarized as follows:



The important difference from Phase 2 is that the DH exchange is no longer performed with an unauthenticated peer.

6. Phase 3 Result

Phase 3 adds server authentication to the secure chat application using PKI.

The client now performs two authentication checks before establishing the encrypted session:

1. **Certificate validation** – verifies that the server certificate was issued by the trusted CA.
2. **Proof-of-possession** – verifies that the server actually possesses the private key corresponding to the certificate.

The legitimate server passes both checks and is then allowed to proceed with the DH exchange.

The modified MITM proxy was tested in two ways:

- Using an untrusted certificate – rejected during certificate validation.
- Using the legitimate certificate without the corresponding private key – rejected during proof-of-possession.

Therefore, the Phase 2 MITM attack is no longer successful against the Phase 3 authentication mechanism.

The main security improvement introduced in this phase is that the client can now **authenticate the server before trusting the DH key exchange**.

Phase 4 – End-to-End Encryption Between Clients

1. Overview

Phase 4 adds an additional end-to-end encryption layer between clients.

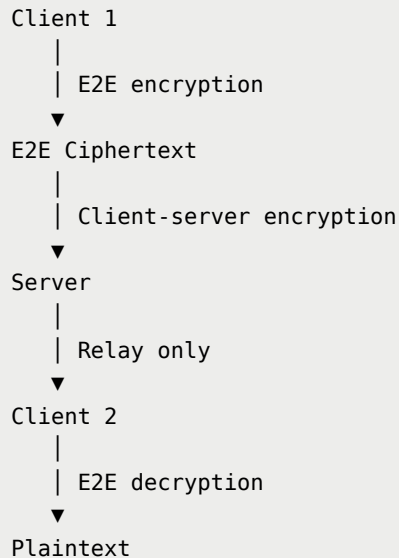
When Client 1 executes:

```
/e2e client2
```

Client 1 and Client 2 perform a separate Diffie-Hellman exchange directly at the application level. The server only relays the

handshake messages and does not obtain the resulting shared key. Once the E2E session is established, chat messages are encrypted using the client-to-client key before being sent through the existing encrypted client-server connection from Phase 3.

The resulting structure is:



The server can still see routing information such as the sender and destination username, but it cannot derive the E2E key or decrypt the message content.

2. E2E Key Establishment

The `/e2e username` command triggers the client-to-client key exchange.

The protocol distinguishes E2E handshake messages from normal chat messages using tags such as:

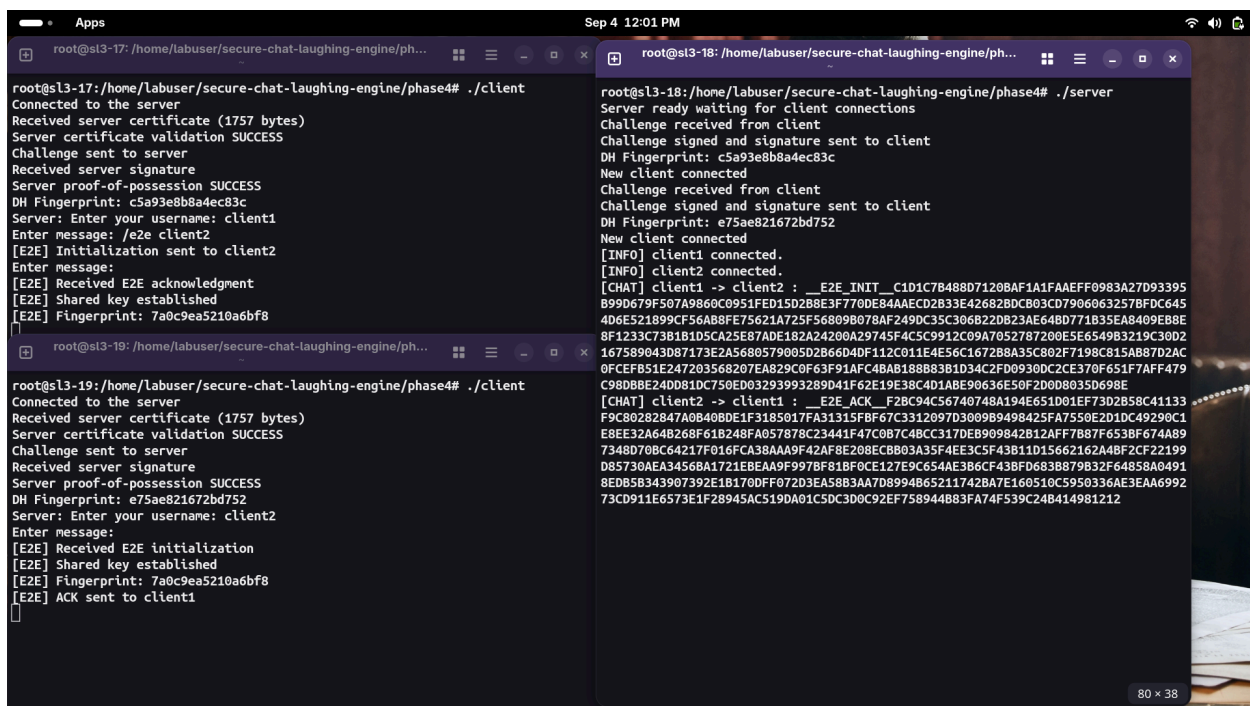
```
__E2E_INIT__  
__E2E_ACK__
```

__E2E_MSG__

The server does not need to understand these messages cryptographically. It simply forwards them to the appropriate client.

After the exchange, both clients independently derive the same E2E key.

Figure 10 – Matching E2E Fingerprints



The image shows three terminal windows from a macOS desktop environment. The top-left window (root@sl3-17) shows a client connecting to a server, receiving a certificate, and sending a challenge. The top-right window (root@sl3-18) shows the server waiting for connections, receiving a challenge from the client, and sending a signed challenge back. The bottom window (root@sl3-19) shows a second client connecting, receiving a certificate, and sending a challenge. The server's response in the top-right window includes a long DH fingerprint string that matches the fingerprint shown in the bottom window, confirming that both clients have derived the same E2E key.

```
root@sl3-17: /home/labuser/secure-chat-laughing-engine/phase4# ./client
Connected to the server
Received server certificate (1757 bytes)
Server certificate validation SUCCESS
Challenge sent to server
Received server signature
Server proof-of-possession SUCCESS
DH Fingerprint: c5a93e8b8a4ec83c
Server: Enter your username: client1
Enter message: /e2e client2
[E2E] Initialization sent to client2
Enter message:
[E2E] Received E2E acknowledgment
[E2E] Shared key established
[E2E] Fingerprint: 7a0c9ea5210a6bf8

root@sl3-18: /home/labuser/secure-chat-laughing-engine/phase4# ./server
Server ready waiting for client connections
Challenge received from client
Challenge signed and signature sent to client
DH Fingerprint: c5a93e8b8a4ec83c
New client connected
Challenge received from client
Challenge signed and signature sent to client
DH Fingerprint: e75ae821672bd752
New client connected
[INFO] client1 connected.
[INFO] client2 connected.
[CHAT] client1 -> client2 : __E2E_INIT_C1D1C7B488D7120BAF1A1FAAEFF0983A27D93395
B99D679F507A9860C0951FED15D2B8E3F770DE84AAEC02833E42682BDCB03CD7906063257BFDC645
4D6E521899CF56AB8FE75621A725F56809B078AF249DC35C3068220B23AE64BD771B35EA8409E88E
8F1233C73B181D5CA25E87ADE182A24200A29745F4C5C9912C09A7052787200E5E654983219C3802
167589043D87173E2A5680579005D2B66D4DF112C011E4E56C1672B8A35C802F7198C815A887D2AC
0FCEFB51E247203568207EAB29C0F63F91AFC48AB188883B1D34C2FD0930DC2CE370F651FAFF479
C98DBBE24DD81DC750ED03293993289D41F62E19E38C4D1ABE90636E50F2D0D80350698E
[CHAT] client2 -> client1 : __E2E_ACK_F2B8C94C56740748A194E651D01EF73D2B58C41133
F9C80282847A0B40BDE1F3185017FA31315FBF67C3312097D3009B9498425FA7550E2D1DC49290C1
E8EE32A648268F61B248FA057878C23441F47C0B7C48CC317DEB909842B12AFF7B87F653BF674A89
7348D70BC64217F016FCA38AAA9F42AF8E208ECB03A35F4EE3C5F43B11D15662162A48F2CF22199
D85730AEA3456BA1721EBEAA9F997BF818F0CE127E9C654AE3B6CF43BF683B879832F64858A0491
8ED5B343907392E1B1700FF072D3EA58B3AA7D8994865211742BA7E160510C5950336AE3EAA6992
73CD911E6573E1F28945AC519DA01C5DC300C92EF758944883FA74F539C24B414981212

root@sl3-19: /home/labuser/secure-chat-laughing-engine/phase4# ./client
Connected to the server
Received server certificate (1757 bytes)
Server certificate validation SUCCESS
Challenge sent to server
Received server signature
Server proof-of-possession SUCCESS
DH Fingerprint: e75ae821672bd752
Server: Enter your username: client2
Enter message:
[E2E] Received E2E initialization
[E2E] Shared key established
[E2E] Fingerprint: 7a0c9ea5210a6bf8
[E2E] ACK sent to client1
```

The matching fingerprints provide evidence that both clients derived the same E2E key.

3. End-to-End Message Encryption

After the E2E session is established, messages are encrypted using the client-to-client key and AES-GCM.

The resulting ciphertext is wrapped using:

__E2E_MSG__

and sent through the already encrypted client-server connection. The server therefore receives only opaque E2E ciphertext.

Figure 11 – Server Relaying E2E Ciphertext

The server can identify that `client1` is sending data to `client2`, but the actual message content is not visible.

For comparison, before an E2E session is established, the server can still read normal chat messages, consistent with the Phase 2/3 design.

4. End-to-End Verification

A test message was sent from Client 1 to Client 2 after establishing the E2E session.

The server log showed only the E2E ciphertext, while Client 2 successfully decrypted and displayed the original message.

Figure 12 – E2E Message Successfully Decrypted

```
root@sl3-17: /home/labuser/secure-chat-laughing-engine/phase4# ./client
Connected to the server
Received server certificate (1757 bytes)
Server certificate validation SUCCESS
Challenge sent to server
Received server signature
Server proof-of-possession SUCCESS
DH Fingerprint: c5a93e8b8a4ec83c
Server: Enter your username: client1
Enter message: /e2e client2
[E2E] Initialization sent to client2
Enter message:
[E2E] Received E2E acknowledgment
[E2E] Shared key established
[E2E] Fingerprint: 7a0c9ea5210a6bf8
hello client2 how are you
Enter message:

Server proof-of-possession SUCCESS
DH Fingerprint: e75ae821672bd752
Server: Enter your username: client2
Enter message:
[E2E] Received E2E initialization
[E2E] Shared key established
[E2E] Fingerprint: 7a0c9ea5210a6bf8
[E2E] ACK sent to client1

[E2E MESSAGE] hello client2 how are you
[]

root@sl3-18: /home/labuser/secure-chat-laughing-engine/phase4# ./server
Server ready waiting for client connections
Challenge received from client
Challenge signed and signature sent to client
DH Fingerprint: c5a93e8b8a4ec83c
New client connected
Challenge received from client
Challenge signed and signature sent to client
DH Fingerprint: e75ae821672bd752
New client connected
[INFO] client1 connected.
[INFO] client2 connected.
[CHAT] client1 -> client2 : _E2E_INIT_C1D1C7B488D7120BAF1A1FAAEFF0983A27D93395
399D679F507A9860C0951FED15D2B8E3F770DE84AAECD2B33E42682BDC803CD7906063257BFD6C45
4D6E521899CF56AB8FE75621A725F56809B078AF249DC35C306B220B23AE64BD771B35EA8409EB8E
3F1233C73B1B1D5CA25E87ADE182A24200A29745F4C5C9912C09A7052787200E5E6549B3219C30D2
167589043D87173E2A5680579005D2B66D4DF112C011E4E56C167288A35C802F7198C815AB87D2AC
3FCFB51E247203568207EA829C0F63F91AFC4BAB188B83B1D34C2FD0930DC2CE370F651F7AFF479
C98DBBE24DD81DC750ED03293993289D41F62E19E38C4D1ABE90636E50F2D0D80350698E
[CHAT] client2 -> client1 : _E2E_ACK_F28C94C56740748A194E651D01EF73D2B58C41133
F9C80282847A0B40BDE1F3185017FA31315FBF67C3312097D3009B9498425FA7550E2D1DC49290C1
EBEE32A648268F61B248FA057878C23441F47C0B7C4BCC317DEB909842B12AFF7B87F653BF674A89
7348D70BC64217F016FCA38AA9F42AF8E208ECBB03A35F4EE3C5F43B11D15662162A4BF2CF22199
D85730AEA34568A1721EBEAA9F997BF81BF0CE127E9C654AE3B6CF43BF0683B879B32F64858A0491
8EDB5B343907392E1B170DFF072D3EA58B3AA7D8994865211742BA7E160510C5950336AE3EAA6992
73CD911E6573E1F28945AC519DA01C5DC3D0C92EF758944883FA74F539C248414981212
[CHAT-E2E] client1 -> client2 : _E2E_MSG_C85E3B92559A3F39E66D025BF3DA7DE09B23
8EDB2A6A7C7151FCEBA916923773BD80D09C0855860C6712031D7AC8559888B133F5F
[]
```

- This confirms that:
- Client 1 and Client 2 successfully established a shared E2E key.
 - The server could not read the E2E message content.
 - The server continued to function purely as a relay.
 - Client 2 could correctly decrypt the message.

5. Phase 4 Result

Phase 4 introduces an additional encryption layer between clients without replacing the existing client-server security from Phase 3.

The server continues to provide routing and transport, but it does not possess the client-to-client E2E key. The matching fingerprints demonstrate that the two clients independently derived the same key, while the server log demonstrates that E2E chat content is transmitted only as opaque ciphertext.

Thus, the confidentiality of messages is extended from client-server encryption to true client-to-client end-to-end encryption.

Phase 5 – Forward Secrecy via Key Rotation

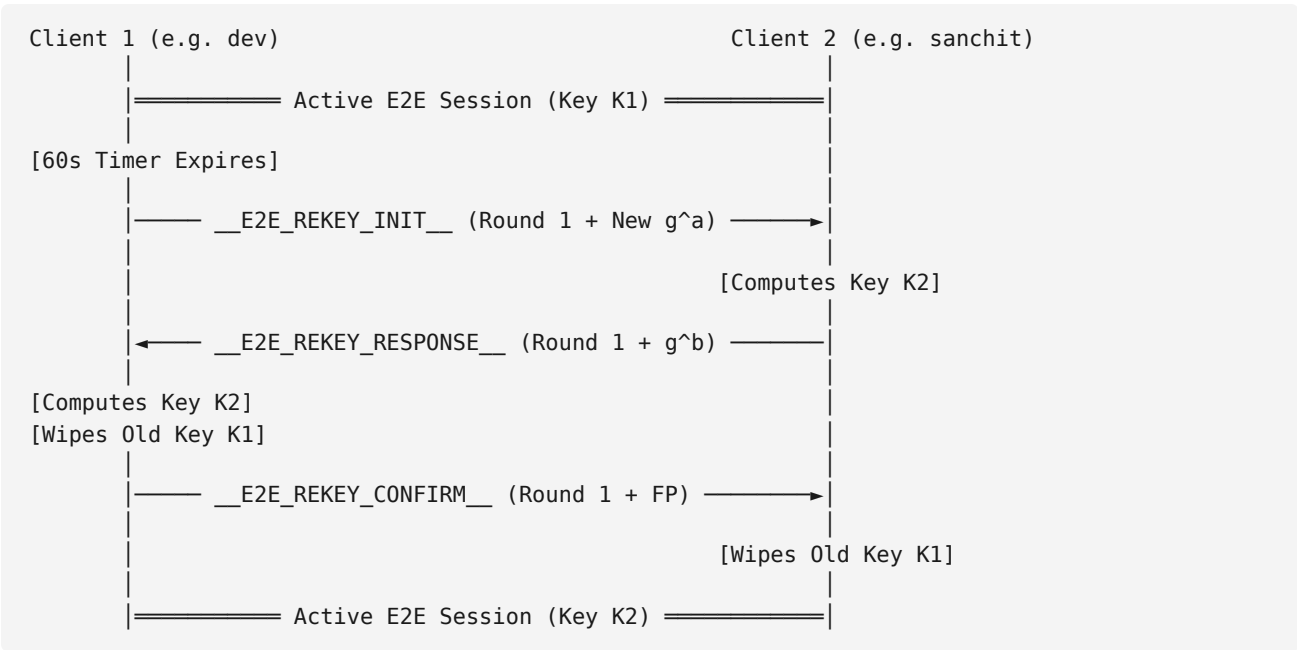
1. Overview

While Phase 4 provides end-to-end confidentiality between clients, it relies on a single, long-lived session key negotiated at connection time. If an adversary passively logs all encrypted network traffic over the wire and later compromises one of the endpoints (or extracts the key from memory), the adversary can retroactively decrypt all historical messages.

Phase 5 addresses this limitation by introducing **Forward Secrecy via periodic key rotation**. The shared symmetric key between Client 1 and Client 2 is automatically renegotiated on a fixed timer every 60 seconds. Each renegotiation produces a genuinely new, independent key, and the previous key is discarded and erased from memory.

2. Automatic Key Rotation Architecture

Key rotation is managed automatically by a dedicated background thread (`rekeyTimer`) running on each client:



For every rotation cycle:

1. A fresh ephemeral Diffie–Hellman key pair is generated using RFC 3526 Group 14 (`new DHE()`, `generateKeys()`).
 2. The public key is exchanged inside an encrypted control message relayed through the server.
 3. Both clients compute the new shared secret, hash it with SHA-256 to derive a new 256-bit AES key, and verify matching fingerprints.
 4. The previous key is securely wiped and freed from memory using `destroyDHE()`.
-

3. Forward Secrecy and Old Key Destruction

To guarantee forward secrecy, old keys must not persist in process memory. The `destroyDHE` function explicitly clears the OpenSSL `BIGNUM` private and public components using `BN_clear()`, zeroes out the raw AES symmetric key vector with `std::fill`, and frees the allocated memory:

```
void destroyDHE(DHE*& dh) {
    if (dh != nullptr) {
        BN_clear(dh->priv_key);
        BN_clear(dh->pub_key);
        std::fill(dh->aes_key.begin(), dh->aes_key.end(), 0);
        delete dh;
        dh = nullptr;
    }
}
```

Once confirmation completes, the old session key is permanently erased from memory.

4. Collision Avoidance Mechanism

A critical challenge in decentralized rekeying is handling race conditions where both clients attempt to trigger renegotiation at approximately the same time. If both clients simultaneously send rekey initialization requests, they risk computing mismatched keys or deadlocking.

To resolve this, the protocol enforces a deterministic leader election based on lexicographical username comparison:

```
if (localUsername >= e2ePartner) {
    return; // Only the client with lower username initiates rotation
}
```

By designating the client with `localUsername < e2ePartner` (e.g. `dev < sanchit`, where `dev` initiates rotation) as the authoritative rotation initiator,

collisions are eliminated by design. If an unexpected rekey initialization arrives while one is already in progress, the lower-priority peer yields, drops its pending request, and answers the initiator's request.

5. Verification and Results

5.1 Successive Key Rotations (Fingerprints and Timestamps)

The chat application was deployed and verified across separate virtual machines:

- **Server VM (server):** 10.91.240.119:1111
- **Client 1 VM (client1):** 10.91.240.136 (User: dev)
- **Client 2 VM (client2):** 10.91.240.179 (User: sanchit)

The verification demonstrated the following lifecycle across two full rotations:

- **Initial Handshake:** Both client VMs established the initial E2E session across the virtual network and printed matching fingerprints:

```
[E2E] Fingerprint: 036f6ef30aa52e33
```

- **Pre-Rotation Chat:** Client 1 (dev) transmitted "Hello sanchit how are you ?" , which Client 2 (sanchit) received and decrypted successfully.
- **60-Second Rotation (Round 1):** After 60 seconds, the background timer automatically triggered rotation. Both clients renegotiated keys and confirmed matching fingerprints at the exact same timestamp:

```
[E2E] Key rotation completed at 2026-09-04 23:23:58 (Round 1)
[E2E] New fingerprint: c2d66cf834bb1a56
```

- **60-Second Rotation (Round 2):** After another 60 seconds, the background timer triggered a second rotation. Both clients renegotiated keys again and confirmed matching fingerprints at the exact same timestamp:

```
[E2E] Key rotation completed at 2026-09-04 23:24:58 (Round 2)
[E2E] New fingerprint: deff0e18ea3c60e3
```

The fingerprints changed from 036f6ef30aa52e33 to c2d66cf834bb1a56 to deff0e18ea3c60e3 , proving across two successive rotations that:

1. The fingerprint changes each time (genuinely fresh ephemeral secrets).
2. Both clients agree on the exact same fingerprint after every rotation.

Figure 13 – Matching Phase 5 Rekey Fingerprints Across Rotations

```
ubuntu@server:~/chat-app/phase5$ ./server
Server ready waiting for client connections
Server IP: 127.0.0.1
Server Port: 1111
Challenge received from client
Challenge signed and signature sent to client
DH Fingerprint: 530186ac23dc7148
New client connected
[INFO] dev connected.
Challenge received from client
Challenge signed and signature sent to client
DH Fingerprint: c3f694390f34dc94
New client connected
[INFO] sanchit connected.
[CHAT] dev -> sanchit : E2E_INIT 808BF5624806F143F192942ED07D6ECB81
14A63395C509D8A80F9615A57D89457B0A1C67492778D20B8DCE7C507192B98884841
B8F9742D11C5D78AC7F482A530B8C77E23C940BF46527232D6A48931348011150
4994FC37A381B3B8A00C9B814C80B8DC76147C42B0D1EBC9A98AC8B94888C5084FF
799C3E83608308310X2CF3BFDD386889438EBC38978A38E2E3556F1E6A3E39A234722
82765557807941841D12C7A8088FCC2D0A8B844E8E8F925C7A8A8888881988999
18010888997624215A38A114C0E65C38DF25F406A34E1FDF31881402777E40A003
846201A1225414886F8FF3F44A05E1846596468286C5158C
[CHAT] sanchit -> dev : E2E_MSG F0C35744E4816DA167936ED6A0CEE29C404
306931AA01332981A3A10528F9C4C606F1798325E8012048F01834871301B1
D1E37A76D03E0F6F28D27861E0E2E36E6E995C1678D5192853D3CD3A1655D1EAD0
5F0999369777E488E827CD01C62E8930835006F150788F22FE8C6170AEDC72E584A
A53657EC2C324F8777065A058F34E0C8188C35405834E7C5AD6F159780445E7FE2
648C4F8564C43F40808E97708E994A9903FF4922980513F031322824C49405
97074048CL92064091E77F1A68E89C38C0A1177B831C59C48A8284F30F7A86F36D2
6A7A68C79897889147F308FC8AC2724E4C43827ECS1578
[CHAT-E2E] dev -> sanchit : E2E_MSG 80226800831A72C74C423483090808
28D2C3E1D0885304F1367083148D988081C1D08A4E0D18BF832184028805A8F83C8
4F7A852
[CHAT-E2E] dev -> sanchit : E2E_MSG 83EEFF7389F829D060254F873714F58
B099864F0743C05F738A55348A47F7580D784C78C8C744603A7863448A874
4FA80071CF7434D7838AF3A08C8F8C798112A0258E32AFB93F8F869E786421A8CF6
A1EC34CE2A56C772951E1A9F07375A4776177843E06780D7187F96F7A0A5558F34
C4F4A20B174314E7C748A0A90F1C20E2F5C27E831765718EEED109731374F05
0A833C86AC2B1A5008041581E488D46631A1C03274C56A7A77F68809F8A311808D5
A3D0963797F06188848A088334FC397544338BF6622C762E91177488AF8A08878C4C
766048C686E19C596FC70872506E598644C25D03194D63E3928A47E143CC8A85878F
295300359C5059155480214086C2762E009532504F7371874E26026A498948
376AN0S32E09765FC84229642AEE4C71E0958C902E2806818FFD1684199507784501
8FCSA31D7A23348A0655959193A0739A27018CB5D8089E89186119F20270861862FE
LAC21A1E2A074A86786135C4970A088621154586012C7195F C83686F94C47C8753CF
81A30511886186248093C9C1C87880228782C5483A54C81BC1F4834591FC8A401
D847228572218275F43A46097623FA0315829088E78C4CAAF8EC16658F3C408FF16
73D060119C3086AFD11017842879CFBF719A8549508B87A58F38D15E8195F8BD7FE5
9C016F8F018E3E3F4E4A1113F838E49F2A51E3D3F7141A17C01C34828CF73B75
D301F808811CD4F8372DC77
[CHAT-E2E] sanchit -> dev : E2E_MSG 8993DFB3CADC6CDD061978F8123E16
7879871868F846F6F42569783D3726F85D0D24ECCE18A154857E44F088866FFAA
40882A024331C156844159E75C402C5AC6871C708F88F88C8C38847F8031867
EEF486C261F996295E832E390B28F83ED787065DC1D603568F8FF3888035062562A0
5065F6D2E8E45473E2FC8D8688F8219994680760848341198E8EF83C2C30F3341
8A027F786822C4C05A8731886C23A0A19F3F3B1A6C6827F9995AC4E476F66A1
8E61291582154A59123A3384FCC572203759E8A5D089F56F96A7F45C5708E0A9C48145
1E0B564C82202C62A088959CAB29E0873C1104E661C751AC9C906049F3A291C04
```

Dual 60-second rekey rotations showing matching fingerprints on dev (middle) and sanchit (right) at 23:23:58 and 23:24:58, while the server (left) relays opaque ciphertext.

5.2 Post-Rotation Message Delivery

To confirm that ongoing communication was not disrupted by key rotation, a message was sent across the virtual network immediately after the Round 1 rotation completed:

- sanchit (on client2 VM) sent: "@dev hello dev, Im good how about you ?"
- dev (on client1 VM) received and decrypted: "[E2E MESSAGE] hello dev, Im good how about you ?"

The server log on the server VM confirmed that it only relayed opaque ciphertext ([CHAT-E2E] sanchit -> dev : E2E_MSG 8993DFB3CADC6CDD...) without accessing message content or key material.

Figure 14 – Successful Post-Rotation Message Delivery Across VMs

```

ubuntu@server:~/chat-app/phase5$ ./server
Server ready waiting for client connections
Server IP: 192.168.1.1
Server Port: 1111
Challenge received from client
Challenge signed and signature sent to client
DH Fingerprint: 5381866c3dc7148
New client connected
[INFO] dev connected
Challenge received from client
Challenge signed and signature sent to client
DH Fingerprint: c3f694390f34dc94
New client connected
[INFO] sanchit connected.
[CHAT] dev -> sanchit : [E2E INIT 088BF5624068f143f12942ED07D6CDB81
14A63395C506D6A80AF9615A57D895F8A01C67492778D20B8DCE1C507192B988884041
B091242D11C579AC7F489A5302B8C77E2C3940BF4652712D2A49313404011150
4994EC32A381E380A06C38E14C80B8DC76147C42B0D1EBC9A98DC8094888C5004FF
799C3E83808380310X2CF38FD380889A38EBC38978A38E2E3556F1E6A3E39A234722
82765357407941841D12C40688FCC2D4B8844E8E8F925C7A8A88E88F1988999
1E010888976742D15A38A314C0E65C38DF25F406A34E1FD5F318814077E40A003
846201A1225414886F8F5F94A4D5EE1846596468286C5158C
[CHAT] sanchit -> dev : [E2E ACK F0C35744E4816DA167936ED6A0EE29C404
306931AA01332981A3A512628F9AC606F81728325E8012048FD1854837130181
D1E3C7AE6D3E0F6F28D27861E0E2E36E6E995C1678D5192853D3CD3A341655D1E400
5FF099036977E488E8272CD01C62E930835006F150788F22F8E65170AEDC72E584A
A536E7C2C32E4F87770654D58FA34E0C188C35405834E7C5A06F15978045E7FE2
64C4F4F64C43F40889537788995A99903F492298051F63143228A24C9405
97074048CL920640917C77F1A68E89C38C0A1177B81C59C48A8584F38F7A86F636D2
6A7A68F389788941473508FCBAC2724E4C43827EC51578
[CHAT] dev -> sanchit : [E2E MSG 682264000931A72C74C42343090908F
28D2C3E1D088530AF1367083148D988081CC1D0884E0188F832184028805A8F83C8
4F7A852
[CHAT] dev -> sanchit : [E2E MSG 63EEFF7389F8298D000254F873714F58
B099884F407143C7F38A5558A847F800784C87C8C744603A78634468A74
4FA90071CF743407838AF30A00CF8C78811A0258E32ABF93F8F8E6E786421A8CF6
A1EC34CE2A56772951E1A9F07375A4776177843E08780718796F7A0A5558F34
C04FA208174314E1CF48A8A8F7A7C208F8E2C07E217657186E2D10973137F65
0A833C86ADC081A5008041581E488D46631A1C03272C56A7A77F6889F9A311808E05
A3D0963797D6188848A08833AF397544338BF662C762E911F7488AF8A08878C4C
766048C68E19C596FC708725066E598644C2503194063E39284A7E143CCAB65878F
2953080350C509165A8014088C2762C08935250497278774E50050499A08
376A8032E09765FC84229642AEE4CF1E0958C902E2806818F01684199507784501
8FCA341D7A233484065595193A0739A27018CB45D8089C89186119F20270861862FE
DA231A1E2A07486F646135C497080861154586012C7159F C83686F94C47C8753C7
81A3C64188618824209302AC18878802278A2C5F85AC4D18BC1F483A591C38A401
D8F4872385722182F5FA34E6097623FA013292988E78C4CAAF8EC16658FCB408FF16
73D060119C308EAF011017842879CFBF719A854505B8F7A58F38D15E8195F88D77E5
9C016F80F18E3E3F4E4F4C113F838E49F2A513E3D3F7141A1C7AD1C34828CF73B75
D3D1F808811CD4F8372DC77
[CHAT] dev -> dev : [E2E MSG 893D8F8C40C6DC0D61978F8123E16
7879871868F84E46F42569783D3726F83D024ECCE18A154857E44F08886FFAA
40882A02A831C1168444759E75C402C5FAC6B7CC0F88F8F8C838478031867
E8F486C261F09629583E3E90828F83ED787065D7CD603568F8F3888035062562A0
506E5F62EEA5473E32FC8D8688F8219996480760848341198E8E8F83C230F3341
B0237F886D23C40504F71886C023A0A19F3F3B14C6827F2999AC46476F66A1
BE51291582154A59133A384FCC572203759E8A5D089F56F96AFF45C5708E80A9C48145
1E08564CB2202C62A88959FCAB29ED0873C1104E661C751AC93906049F3A291C04

```

Live post-rotation message delivery verified across VMs: client 2 transmits message after Round 1 rekey, client 1 decrypts successfully, and server only relays ciphertext.

The message was decrypted without errors or dropped packets, proving that the live chat conversations seamlessly across key boundaries over a live network.

6. Discussion: Forward Secrecy Security Properties

In terms of attacker capabilities, forward secrecy provides the following concrete security guarantees:

- **What an attacker CANNOT do if a key is compromised:**
If an adversary compromises one of the endpoint machines and extracts the active symmetric key K_i , the adversary can only decrypt messages transmitted during that specific 60-second window.
- **What an attacker CANNOT do:**
 1. **Past Traffic (Forward Secrecy):** The adversary cannot decrypt any past messages sent using keys $K_1, K_2, \dots, K_{i-1}$. Because old keys are wiped from memory via `destroyDHE()` and each DH exchange uses fresh random exponents

(a_i, b_i) , knowing K_i provides zero mathematical advantage in deriving past shared secrets.

2. **Future Traffic (Post-Compromise Security):** Once the next 60-second rotation occurs, fresh ephemeral exponents are generated. Unless the adversary maintains persistent, continuous code execution on the host, they cannot decrypt subsequent sessions.

Phase 4 alone lacked this guarantee: a compromise of the single session key in Phase 4 exposes the entire communication history of that session. Phase 5 confines the blast radius of any key compromise to a single 60-second time slice.

Appendix A – Generative AI Usage

Some of the prompts that were used during the development of the assignment:

Phase 1 – Baseline Chat Application

- Explain how to implement a one-to-one chat application using TCP sockets in C++.
- Help me understand how the client and server should communicate in a TCP chat application.
- Help me fix this problem in TCP socket chat application.

Phase 2 – Diffie–Hellman and AES-GCM

- Explain how Diffie–Hellman key exchange works and how it can be implemented using OpenSSL BIGNUM.
- Help me implement Diffie–Hellman manually using RFC 3526 Group 14 without using OpenSSL's built-in DH/ECDH APIs.
- How can I derive an AES-256 key from the Diffie–Hellman shared secret using SHA-256?
- Help me implement AES-256-GCM encryption and decryption using OpenSSL.
- Help me implement a MITM proxy to demonstrate the vulnerability in the unauthenticated Diffie–Hellman exchange.
- Help me troubleshoot AES-GCM decryption and tamper detection.

Phase 3 – PKI and Server Authentication

- Explain how to create a Certificate Authority and issue a server certificate using OpenSSL.
- Help me implement server certificate validation in the C++ client using OpenSSL.
- How can I verify that a server possesses the private key corresponding to its certificate?
- Help me implement a challenge-response proof-of-possession mechanism using OpenSSL signatures.
- Help me update the MITM attack so that it fails against certificate validation and proof-of-possession.
- Help me troubleshoot certificate validation and signature verification errors.

Phase 4 – End-to-End Encryption

- Explain how to modify the existing chat application to provide end-to-end encryption between clients while the server only relays encrypted messages.

- Help me implement a separate Diffie–Hellman exchange between two clients while maintaining the existing client-server encrypted connection.
- How should E2E_INIT, E2E_ACK, and E2E_MSG messages be handled?
- Help me troubleshoot why the E2E acknowledgement is not being received by the initiating client.
- Why can simultaneous E2E key exchanges cause the two clients to derive different keys?
- Review my client.cpp and identify the issue with the E2E handshake.
- Can the E2E implementation be fixed without introducing TCP message framing?

Phase 5 – Forward Secrecy via Key Rotation

- How do I implement periodic key renegotiation in a chat application using a background timer thread?
- How to avoid collisions when both clients attempt to initiate key renegotiation at the same time?
- What is the difference between forward secrecy and break-in recovery (post-compromise security)?
- How should old keys be securely erased from memory in OpenSSL (using BN_clear and vector zeroing)?
- How to ensure that chat messages sent during or right before a key rotation are not dropped?

Troubleshooting and Testing

- Help me troubleshoot this OpenSSL compilation error: fatal error: openssl/bn.h: No such file or directory.
- Help me troubleshoot SSH key and host-key issues on the lab machines.
- Help me understand what the Wireshark packet capture demonstrates about confidentiality.
- What evidence should I include in the report to demonstrate that the MITM attack succeeds or fails?
- Review my implementation and suggest what should be tested for each security phase.

Report Preparation

- Help me structure the Network Security assignment report.
- Rewrite my Phase 1 implementation description in a clear technical style.
- Help me explain the security improvements between each phase.
- What screenshots and evidence should be included in the report?
- Help me write the discussion of security properties and limitations.